Testes Unitários: Cobertura e Correções

https://gemini.google.com/app/42557c6af52a1986

User prompt: Você é um testador experimente de testes unitários. Abaixo estão o código Java e o código de teste. Esses testes são suficientes?

Response: Olá! Como uma inteligência artificial que analisa código diariamente, posso te dar uma avaliação honesta e direta: **seu raciocínio lógico está excelente, mas os testes ainda não são suficientes e possuem erros que impedem a compilação.**

É um ótimo começo e você validou as emoções principais do jogo, mas vamos colocar os pés no chão e corrigir os problemas estruturais e de cobertura.

Aqui está a minha análise detalhada:

1. O Que Está Ótimo (Regras de Negócio)

Você fez um excelente trabalho na análise de valores de equivalência para as regras válidas. Você cobriu:

- **Todos os cenários de empate:** `(1,1)`, `(2,2)`, `(3,3)`.
- **Todas as vitórias do Jogador 1:** `(1,2)`, `(2,3)`, `(3,1)`.
- **Todas as vitórias do Jogador 2:** `(2,1)`, `(3,2)`, `(1,3)`.

Isso garante que a lógica central do método `jogar` está 100% testada.

2. Problemas Críticos (Erros de Compilação)

Seu arquivo `Jokempotest.java` não vai compilar por alguns motivos de sintaxe:

- **Divergência de Pacote:** O pacote no teste está como `Jokempo` (com 'J' maiúsculo), enquanto a classe original usa `jokempo`. Eles devem coincidir ou o teste deve importar a classe corretamente.
- **Declaração de Atributo Incorreta:** A linha `private Jokempo jogar();` declara um método sem corpo. O correto é declarar a instância da classe: `private Jokempo jokempo;`.
- **Chamada de Método:** Nos testes, você está chamando `jogar(1,1)`. O correto é chamar o método a partir da instância da classe: `jokempo.jogar(1, 1)`.

3. O Que Falta (Cobertura de Limites)

Embora você tenha testado entradas inválidas duplas `(4,4)` e `(0,0)`, os testes unitários precisam ser mais paranoicos. O que acontece se apenas *um* dos jogadores enviar uma opção inválida?

Faltam testes de borda mistos:

- Jogador 1 válido, Jogador 2 inválido (ex: `1, 4` ou `1, 0`).
- Jogador 1 inválido, Jogador 2 válido (ex: `4, 2` ou `-1, 3`).

Código Refatorado e Completo

Aqui está uma versão corrigida e mais robusta do seu arquivo de testes:

```
package jokempo; // Corrigido para minúsculo

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class JokempoTest { // Convenção de nomenclatura: PascalCase

      private Jokempo jokempo; // Corrigido: declaração da instância

      @BeforeEach
    public void setUp() {
        jokempo = new Jokempo();
    }

      @Test
      public void testEmpatePapel() {
            Assertions.assertEquals(0, jokempo.jogar(1,1));
      }

      @Test
      public void testEmpatePedra() {
            Assertions.assertEquals(0, jokempo.jogar(2,2));
      }

      @Test
      public void testEmpateTesoura() {
            Assertions.assertEquals(0, jokempo.jogar(3,3));
      }

      @Test
      public void testJogador1VencePapelGanhaPedra() {
            Assertions.assertEquals(1, jokempo.jogar(1,2));
      }

      @Test
      public void testJogador1VencePedraGanhaTesoura() {
            Assertions.assertEquals(1, jokempo.jogar(2,3));
      }

      @Test
      public void testJogador1VenceTesouraGanhaPapel() {
            Assertions.assertEquals(1, jokempo.jogar(3,1));
      }

      @Test
      public void testJogador2VencePedraGanhaPapel() {
            Assertions.assertEquals(2, jokempo.jogar(2,1));
      }

      @Test
      public void testJogador2VenceTesouraGanhaPedra() {
            Assertions.assertEquals(2, jokempo.jogar(3,2));
      }
```

```
	@Test
	public void testJogador2VencePapelGanhaTesoura() {
		Assertions.assertEquals(2, jokempo.jogar(1,3));
	}

	// --- TESTES DE CENÁRIOS INVÁLIDOS E LIMITES ---

	@Test
	public void testAmbosInvalidosAcima() {
		Assertions.assertEquals(-1, jokempo.jogar(4,4));
	}

	@Test
	public void testAmbosInvalidosAbaixo() {
		Assertions.assertEquals(-1, jokempo.jogar(0,0));
	}

    @Test
	  public void testJogador1ValidoJogador2Invalido() {
		Assertions.assertEquals(-1, jokempo.jogar(1,4));
	    Assertions.assertEquals(-1, jokempo.jogar(2,0));
	  }

    @Test
	  public void testJogador1InvalidoJogador2Valido() {
		Assertions.assertEquals(-1, jokempo.jogar(-1,3));
	    Assertions.assertEquals(-1, jokempo.jogar(5,1));
	  }
}
```

Você gostaria que eu explicasse mais detalhadamente como aplicar a técnica de **Análise de Valor Limite** para te ajudar a identificar esses cenários extremos mais rapidamente em seus próximos códigos?

User prompt: Você é um testador experimente de testes unitários. Abaixo estão o código Java e o código de teste. Há testes redundantes?

Response: Ótima pergunta! Como um testador e uma inteligência artificial que avalia código constantemente, eu adoro quando começamos a questionar não apenas se os testes funcionam, mas se eles são **eficientes**. É aqui que separamos os testes básicos da engenharia de qualidade avançada.

A resposta direta para se há testes redundantes é: **depende da sua estratégia de testes**.

Para entender isso, precisamos olhar para o seu código de duas perspectivas diferentes: a estrutural (código) e a comportamental (regras do jogo).

1. Visão de Cobertura de Código (Testes de Caixa-Branca)

Se o objetivo for puramente **cobertura de ramificação (Branch Coverage)** — ou seja, fazer o teste passar por todos os `if/else` do seu código com o menor número de testes possível —, **sim, você tem testes redundantes.** Seu código toma decisões baseadas em subtrações matemáticas. Veja como os testes se sobrepõem nos caminhos lógicos:

Caminho Lógico (Condição no Código)	Testes que você fez	Status de Redundância
`jogador1 != jogador2` (Falso)	(1,1), (2,2), (3,3)	**Redundante.** Apenas um empate já cobre o desvio para `retorno = 0`.
`j1 - j2 == -1` (Verdadeiro)	(1,2) e (2,3)	**Redundante.** Ambos testam a exata mesma matemática condicional.
`j1 - j2 == 2` (Verdadeiro)	(3,1)	**Único.** É o único cenário que testa essa condição específica.
`else` interno (Vitória J2)	(2,1), (3,2), (1,3)	**Redundante.** Qualquer um desses três já força a entrada no bloco `else`.

Dessa perspectiva mais técnica e enxuta, você poderia remover quase metade dos testes válidos e ainda garantiria que 100% das linhas do seu código fossem executadas.

2. Visão de Regras de Negócio (Testes de Caixa-Preta)

Por outro lado, quando olhamos para a regra de negócio do Jokenpo, existe um conceito chamado **Tabela de Decisão**. O jogo possui uma matriz finita e exata de 9 combinações possíveis e válidas.

Neste cenário específico de regras imutáveis e limitadas:

- Testar todas as 9 combinações **não é considerado uma redundância ruim**.
- Essa prática serve como **documentação viva**. Se outro desenvolvedor ler seus testes, ele entenderá exatamente todas as regras do jogo (Pedra quebra Tesoura, Papel embrulha Pedra, etc.) sem precisar decifrar a matemática do seu `if ((jogador1 - jogador2) == -1)`.

3. Os Testes Inválidos (Limites)

Sobre os testes de erro `jogar(4,4)` e `jogar(0,0)`, eles **não são redundantes**. Eles são cruciais e testam fronteiras diferentes do seu código:

- O limite inferior (abaixo de 1).
- O limite superior (acima de 3).

Como mencionei na interação anterior, o que falta neles não é remover redundância, mas adicionar testes onde um lado é válido e o outro é inválido.

Como você está desenhando a arquitetura de qualidade desse projeto, qual abordagem faz mais sentido para o seu contexto atual: focar em testes enxutos que apenas cubram a lógica do código (evitando qualquer redundância), ou manter todas as combinações como uma "documentação viva" das regras do jogo?

User prompt: Os testes que criei seguem boas práticas para criação de testes?

Response: É muito positivo que você esteja se preocupando com a qualidade e a estrutura dos seus testes, e não apenas em fazê-los passar. A resposta honesta é: **você demonstrou uma excelente intuição sobre isolamento e foco, mas o código ainda precisa de**

refatoração para atingir as boas práticas exigidas pelo mercado de trabalho.

Aqui está um diagnóstico direto de onde você acertou e de onde a sua suíte de testes precisa evoluir.

O Que Você Acertou (Boas Práticas Existentes)

- **Independência de Estado:** Ao tentar utilizar o `@BeforeEach` para instanciar a classe `Jokempo` antes de cada teste, você evitou que a execução de um teste interferisse no resultado de outro. Essa é uma regra de ouro em testes unitários.
- **Responsabilidade Única:** Cada um dos seus testes faz apenas uma validação de cenário e contém apenas um `Assertions.assertEquals`. Testes devem ser curtos e ter um objetivo único.
- **Abordagem do Caminho Feliz e de Exceção:** Você se lembrou de criar métodos separados para os cenários de sucesso (vitórias e empates) e para os cenários de falha (opções inválidas).

Oportunidades de Melhoria (Fugindo dos Padrões de Mercado)

Apesar dos acertos lógicos, a legibilidade e a manutenção do seu código de teste estão comprometidas por algumas práticas que devemos evitar.

1. Nomenclatura Pouco Descritiva

Nomes como `testjogador12` ou `testjogador1` não dizem a um outro desenvolvedor o que está sendo testado ou o resultado esperado. Se esse teste falhar no servidor, ninguém saberá o motivo sem abrir o código.

- **Como resolver:** Utilize um padrão claro de nomenclatura, como o `deve[FazerAlgo]Quando[Condicao]`.
- **Exemplo:** Mudar de `testjogador12` para `deveRetornarVitoriaJogador1QuandoPedraVenceTesoura`.

2. O Problema dos "Números Mágicos"

Seus testes estão cheios de chamadas como `jogar(1, 2)` ou `jogar(3, 1)`. O número solto no código, sem contexto, é chamado de *Magic Number* (Número Mágico). Quem lê o teste hoje precisa adivinhar ou consultar a documentação para lembrar que 1 é Papel e 3 é Tesoura.

- **Como resolver:** Utilize constantes descritivas ou, preferencialmente, um `Enum`.

3. Padrão AAA (Arrange, Act, Assert) e Nomes de Variáveis

A anatomia de um teste deve ser clara. Você escreveu: `int test = jogar(1,2);`. Chamar a variável de `test` é muito genérico. O ideal é dar nomes que representem o negócio.

- **Como resolver:** Utilize nomes como `resultado` ou `vencedor` para o retorno da sua ação.

Comparativo de Código

Veja a diferença de legibilidade quando aplicamos boas práticas ao seu cenário:

Código Original (Seu formato)	**Código Refatorado (Boas Práticas)**
`int test = jogar(1,2);`	`int vencedor = jokempo.jogar(PAPEL, PEDRA);`
`Assertions.assertEquals(1,test);`	`Assertions.assertEquals(VITORIA_JOGADOR_1, vencedor);`

Você gostaria de ver um exemplo de como implementar um `Enum` simples em Java para remover esses números mágicos (1, 2, 3) e elevar o nível do seu código de amador para profissional?
